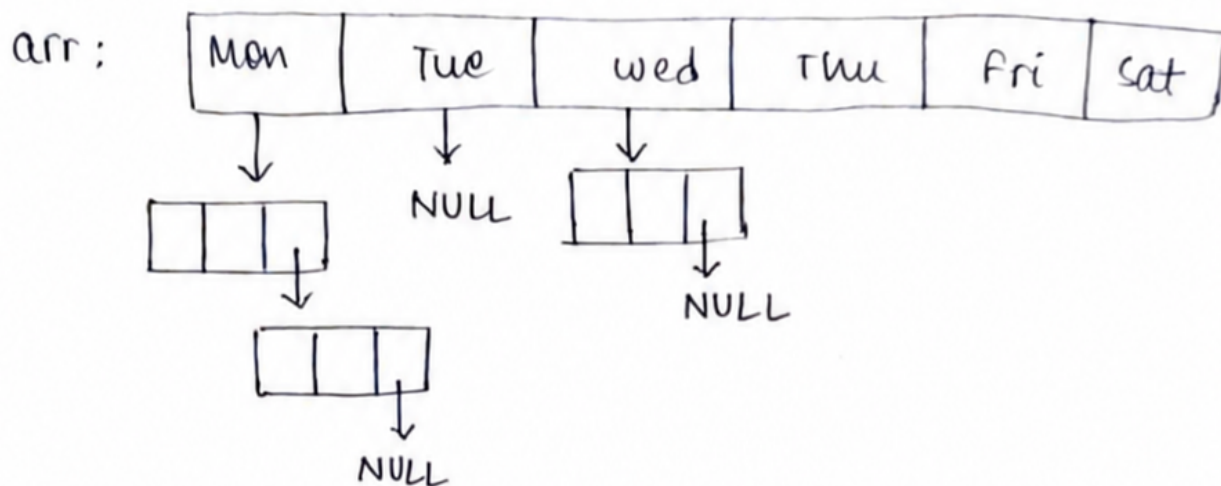


Abhigya Narayan
Awasthi
B11
2501030085

Data Structures Tutorial-6

Ques-1)



```
class Node {
    string slot;
    string subject;
    Node* next;
```

```
public:
    Node(string s, string sub) {
        slot = s;
        subject = sub;
        next = nullptr;
    }
```

```
};
```

⇒ arr : array of Node* pointers
pointing to first class of that
day.

```
void print(intNode* arr[], int n) {
```

```
    string day;
```

```
    if (
```

```
        for (int i=0; i<n; i++) {
```

```
            if (i==0) day = "Mon";
```

```
            else if (i==1) day = "Tue";
```

```
            ;
```

```
            cout << day << endl;
```

```
            if (arr[i]) {
```

```
                while Node* curr = arr[i];
```

```
                while (curr) {
```

```
                    if (curr->subject == "ds") {
```

```
                        cout << curr->slot << " ,
```

```
                        << curr->subject;
```

```
                    }
```

```
                    curr = curr->next;
```

```
                }
```

```
            }
```

```
        }
```

```
    }
```

Ques 2)

~~void~~ Flatten (Node* head) {

Node*

Node* curr = head;

~~while~~ if (curr->child) {

if (!curr->child) curr = curr->next;

else {

Node* temp = curr->next;

curr->next = curr->child;

curr->child = nullptr;

Node* tail = curr->child;

while (tail->next) {

tail = tail->next;

}

// continued normally

tail->next = temp;

curr = curr->next;

}

return head;

}

Ques 3)

```
void TotalLevels (Node * head) {  
    vector < Node * > vec;  
    Node * curr = head;  
    while curr  
    if (!head) return;  
    queue < Node * > q;  
    q.push
```

Ques 3)

```
int count [100]; // to store all numbers of  
                  nodes at each level.
```

```
// Level starting at 1
```

```
int maxlevel = 1;
```

```
void totallevels (Node * head, int level) {
```

```
    Node * curr = head;
```

```
    while (curr) {
```

```
        count[level]++;
```

```
        if (level > maxlevel) {
```

```
            maxlevel = level;
```

```
        }
```

```
        if (curr->child) {
```

```
            totallevels (curr->child, level+1);
```

```
        }
```

```
        curr = curr->next;
```

```
    }
```

```
}
```

```
// in main call totallevels  
    (head, 1);
```

```
int main() {
```

```
    totalLevels(head, 1);
```

```
    for (int i = 1; i <= maxLevels; i++) {
```

```
        cout << "Level " << i << " : " <<
```

```
        count[i] << "Levels\n";
```

```
    }
```

```
    cout << "Max depth : " << maxLevels;
```

```
    return 0;
```

```
}
```

Ques 4).

```
void reverseChild (Node* parenthead) {
```

```
    Node* p = parenthead;
```

```
    while (p) {
```

```
        if (p->child) {
```

```
            Node* c = p->child;
```

```
            reverse(c);
```

```
        }
```

```
        p = p->next;
```

```
    }
```

```
}
```

Node* h.

```
void reverse (Node* childhead) {
```

```
    Node* prev = NULL;
```

```
    Node* curr = childhead;
```

```
    Node* next = NULL;
```

```
while (curr) {
```

```
    next = curr → next;
```

```
    curr → next = prev;
```

```
    prev = curr;
```

```
    curr = next;
```

```
}
```

```
chitohhead = prev;
```

```
}
```